Assignment 6 – Lab - Standard Input, Standard Output, and Standard Error in Linux

The objective of this lab assignment is to gain hands-on experience with Standard Input (stdin), Standard Output (stdout), and Standard Error (stderr) in the Red Hat Enterprise Linux (RHEL) 9 operating system.

This lab will focus on practical exercises to reinforce the understanding of these standard streams.

Access a RHEL 9 terminal for the following exercises.

Questions for this assignment

1. Please provide a practical explanation of Standard Input (stdin), including an example.
2. Please provide a practical explanation of Standard Output (stdout), including an example.
3. Please provide a practical explanation of Standard Error (stderr), including an example.
4. Please provide a practical explanation of Pipelines and Redirection, including an example.

Solution by the Instructor:

1. Please provide a practical explanation of Standard Input (stdin), including an example.
    i. Open the terminal.
    ii. Create a simple script named user_input.sh that prompts the user for their name using read and then echoes a greeting.

```
#!/bin/bash

echo "Enter your name:"

read name

echo "Hello, $name!"
```

iii. Run the script and observe how it interacts with user input.

2. Please provide a practical explanation of Standard Output (stdout), including an example.
   i. Create a script named output_example.sh that generates output to the terminal.
      ```
      #!/bin/bash
      echo "This is an example of standard output."
      ```
   ii. Run the script and redirect its output to a file named output.txt.
      ```
      ./output_example.sh > output.txt
      ```
   iii. View the content of output.txt to confirm the redirection.

3. Please provide a practical explanation of Standard Error (stderr), including an example.
   i. Create a script named error_example.sh that intentionally generates an error and outputs an error message to stderr.
      ```
      #!/bin/bash
      echo "This is a deliberate error" >&2
      exit 1
      ```
   ii. Run the script and redirect its stderr to a file named error.log.
      ```
      ./error_example.sh 2> error.log
      ```
   iii. View the content of error.log to observe the error message.

4. Please provide a practical explanation of Pipelines and Redirection, including an example.
   i. Create a script named pipeline_example.sh that generates some output.
      ```
      #!/bin/bash
      echo "Output from the first command."
      ```
   ii. Create another script named process_output.sh that processes input from stdin.
      ```
      #!/bin/bash
      echo "Processing input from stdin..."
      while read line; do
        echo "Processed: $line"
      done
      ```
   iii. Use a pipeline to connect the output of pipeline_example.sh to the input of process_output.sh.
      ```
      ./pipeline_example.sh | ./process_output.sh
      ```